Week 4 Module:

Date: 09/07/2026

Time & Space Complexity (Asymptotic Analysis):

Objective:
In this week's Module, you will learn how to analyze the efficiency of an algorithm using Time Complexity and Space Complexity. You will also revisit the problems solved in previous modules and determine their complexities.

Topics to Learn & Write in notebook:

- Introduction to Algorithm Analysis
- Time Complexity
- Space Complexity
- Asymptotic Notations
    - Big O (O)
    - Omega (Ω)
    - Theta (Θ)
- Best, Average & Worst Case
- Rules to Calculate Complexity
- Time Complexity of Loops
- Time Complexity of Nested Loops
- Time Complexity of Logarithmic Loops
- Space Complexity Basics

Section A – Complexity Analysis of Previous Modules:

In this section, revisit all the topics and programming problems covered in Module 1, Module 2, and Module 3.
For each topic and question, write:

- Time Complexity
- Space Complexity
- A brief explanation (2–3 lines) justifying why the complexity is correct.
- If an optimized approach exists, mention it.

Note: Do not simply write O(n) or O(1). Your explanation should clearly describe how the algorithm works and why it results in that complexity.

Section B – Analyze the Algorithm Complexity:

Solve each problem using C++/java, then analyze your solution.

For every problem, you must submit:

- C++/java Code
- Time Complexity
- Space Complexity
- Explanation of why the calculated complexities are correct (2–3 lines)
- If a more efficient approach exists, briefly explain it.

Problem Statement:

Given a positive integer N, print all the numbers from 1 to N in increasing order, separated by a single space.

Your task is to write a program that prints the required sequence.

After completing the program, analyze your solution by calculating its Time Complexity and Space Complexity, and explain why your answer is correct.

Input Format

A single integer N.

Output Format

Print all integers from 1 to N in increasing order separated by spaces.

Constraints

$1 \leq N \leq 10^5$

Example 1

Input

5

Output

1 2 3 4 5

Explanation

The numbers from 1 to 5 are printed in increasing order.

Example 2

Input

8

Output

1 2 3 4 5 6 7 8

Explanation

The program prints every integer starting from 1 until 8, maintaining increasing order.

Problem Statement:

Given a positive integer N, find the sum of the first N natural numbers using a loop.

Your task is to write a program that calculates and prints the required sum.

After completing the program, analyze your solution by calculating its Time Complexity and Space Complexity, and explain your answer.

Input Format

A single integer N.

Output Format

Print the sum of the first N natural numbers.

Constraints

$1 \leq N \leq 10^7$

Example 1

Input

5

Output

15

Explanation

The sum of the first five natural numbers is:

1 + 2 + 3 + 4 + 5 = 15

Example 2

Input

10

Output

55

Explanation

The sum of the first ten natural numbers is:

1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 = 55

Problem Statement:

Given two integers A and B, calculate the value of A^B (A raised to the power B) using a loop.

Your task is to write a program that computes the result and prints it.

Once your program is complete, analyze your solution by calculating its Time Complexity and Space Complexity, and explain why your answer is correct.

Input Format

Two integers A and B separated by a space.

Output Format

Print the value of A^B.

Constraints

$1 \leq A \leq 100$

$0 \leq B \leq 20$

Example 1

Input

2 5

Output

32

Explanation

The value of 2⁵ is:

$2 \times 2 \times 2 \times 2 \times 2 = 32$

Example 2

Input

3 4

Output

81

Explanation

The value of 3⁴ is:

$3 \times 3 \times 3 \times 3 = 81$